Exp No: 7	Flow Control at Data Link Layer using Sliding Window Protocol

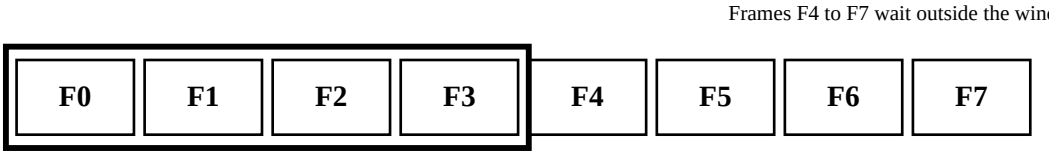
AIM:

To write a program that implements flow control at the data link layer using the Sliding Window Protocol (Go-Back-N) and to simulate the flow of frames from one node to another.

ALGORITHM / PROCEDURE:

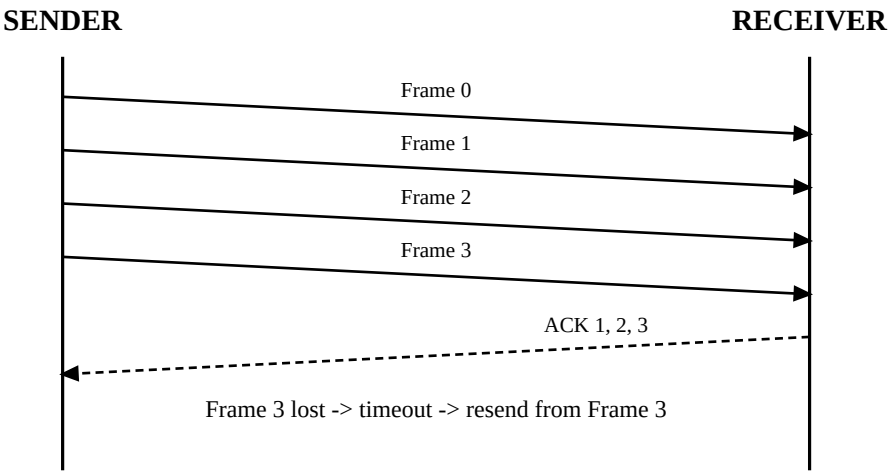
1. Read the total number of frames and the window size N. The sender may send up to N frames without waiting for an acknowledgement.
2. Keep two pointers at the sender: `base` (oldest unacknowledged frame) and `nxt` (next frame to send).
3. While `nxt < base + N`, send frame number nxt and increase nxt.
4. The receiver accepts frames in order. For each correct frame it sends an acknowledgement ACK (frame number + 1).
5. When the sender receives an ACK, the window slides forward by one position (base = base + 1).
6. If a frame is lost the receiver discards the later frames, the sender times out and sends again from the lost frame (Go-Back-N).
7. Repeat until all the frames are acknowledged and display the sequence of events.

Fig 1: Sender window over the frame sequence



Sender window (size = 4)

Fig 2: Frame and ACK exchange between sender and receiver



PROGRAM:

```
# Sliding window protocol (Go-Back-N) simulation
TOTAL, WINDOW = 8, 4
LOST = {3}                      # frame 3 is lost the first time it is sent

base = nxt = 0
lost_once = set(LOST)
print(f"Total frames = {TOTAL}, Window size = {WINDOW}\n")
while base < TOTAL:
    while nxt < base + WINDOW and nxt < TOTAL:
        print(f"Sender   : Frame {nxt} sent   window=[{base}..{min(base+WINDOW,TOTAL)-1}]")
        nxt += 1
    expected = base
    for f in range(base, nxt):
        if f in lost_once:
            print(f"Network  : Frame {f} LOST")
            lost_once.discard(f)
            print(f"Sender   : Timeout for frame {f}, resending from frame {f}")
            nxt = f
            break
        print(f"Receiver : Frame {f} received, ACK {f + 1} sent")
        print(f"Sender   : ACK {f + 1} received, window slides")
        base = f + 1
print("\nAll frames transmitted successfully.")
```

OUTPUT:

Terminal - Sliding window simulation

```
USER   @USER   -VirtualBox:~$ python3 exp7.py
Total frames = 8, Window size = 4

Sender   : Frame 0 sent   window=[0..3]
Sender   : Frame 1 sent   window=[0..3]
Sender   : Frame 2 sent   window=[0..3]
Sender   : Frame 3 sent   window=[0..3]
Receiver : Frame 0 received, ACK 1 sent
Sender   : ACK 1 received, window slides
Receiver : Frame 1 received, ACK 2 sent
Sender   : ACK 2 received, window slides
Receiver : Frame 2 received, ACK 3 sent
Sender   : ACK 3 received, window slides
Network  : Frame 3 LOST
Sender   : Timeout for frame 3, resending from frame 3
Sender   : Frame 3 sent   window=[3..6]
Sender   : Frame 4 sent   window=[3..6]
Sender   : Frame 5 sent   window=[3..6]
Sender   : Frame 6 sent   window=[3..6]
Receiver : Frame 3 received, ACK 4 sent
Sender   : ACK 4 received, window slides
Receiver : Frame 4 received, ACK 5 sent
Sender   : ACK 5 received, window slides
Receiver : Frame 5 received, ACK 6 sent
Sender   : ACK 6 received, window slides
Receiver : Frame 6 received, ACK 7 sent
Sender   : ACK 7 received, window slides
Sender   : Frame 7 sent   window=[7..7]
Receiver : Frame 7 received, ACK 8 sent
Sender   : ACK 8 received, window slides

All frames transmitted successfully.
```

RESULT:

Thus the sliding window protocol was implemented and the transmission of 8 frames from the sender to the receiver was simulated successfully. The lost frame 3 was retransmitted after the timeout and all frames were delivered in order.